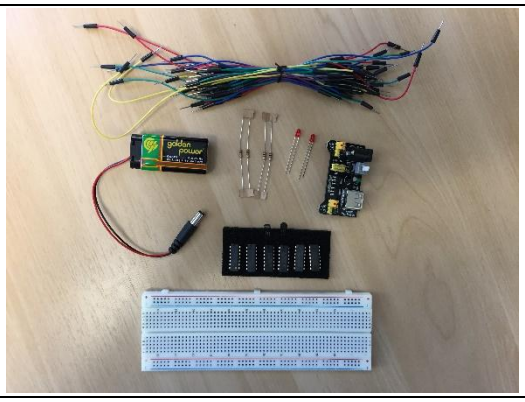
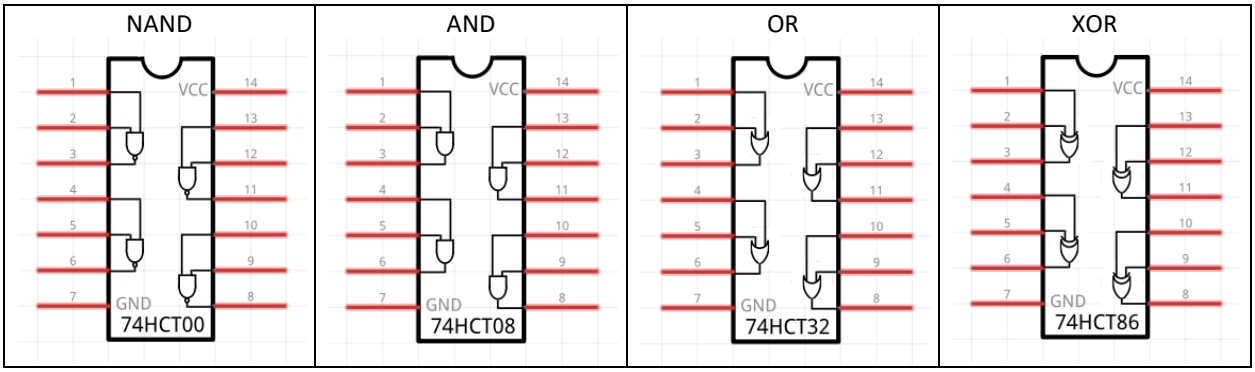
Lab 4

Objective: Use logic chips to perform binary arithmetic.

Each kit should contain:

2 * LED 2 * 1K Ohms resistor (Brown Black Red Gold) 2 * 10K Ohms resistor (Brown Black Orange Gold) 1 * breadboard 1 * power board 1 * 9v battery (and connector) 2 * CTBC 547 JS Transistor (NPN) 2 * 74LS86 (quad 2-input XOR gate) 2 * 74LS08 (quad 2-input AND gate) 1 * 74LS00 (quad 2-input NAND gate) 1 * 74LS32 (quad 2-input OR gate) Bundle of Wire	

The 7400-series logic chips implement four logic gates on a single integrated circuit (IC); hence they are known as *quad gate* chips. The AND, OR, XOR, and NAND chips have similar pinouts:



VCC (collector voltage) connects to the positive power supply rail, and GND (ground) connects to the negative. Without both connections, the chip won't work correctly!

When placing the chips on your breadboard, place them such that the pins straddle the channel / gap in the middle of the breadboard. It's also recommended to use a consistent orientation (with the notches facing the same way).

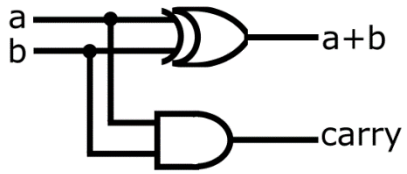
To make the output visible, connect an LED with a 1 kΩ series resistor between the output of a gate and ground. Use jumper wires to connect the gate inputs to either +5V or ground. Note that unlike the transistor-based circuits, these logic chips do not require a series resistor on their inputs. However, you will find that unconnected inputs tend to float high, so you should always connect each input to either +5V or ground when testing (but don't create a short circuit).

Before you start building the circuits, write down the truth tables for the main Boolean logic operators to confirm (or build) your familiarity with them.

Also write the truth table for adding two one-bit binary numbers (with a two-bit result) to help you understand why the half-adder is structured the way it is.

Exercise 1: Half Adder

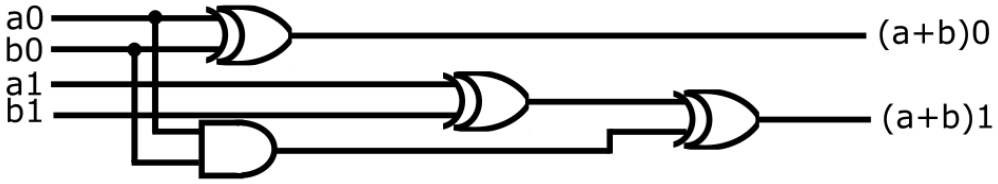
Build a half-adder using the chips and wires from the kit. Recall that, in binary, the least significant bit of *a+b* is given by *a* XOR *b*, and that the *carry*, indicating whether we overflowed the first bit, is *a* AND *b*. Therefore, to build a circuit that adds two 1-bit numbers together and produces a 2-bit answer we only need 2 gates. The circuit looks like this:



Exercise: 2-bit Adder

The circuit for a half adder takes two bits as input and gives a one-bit result with carry. The circuit for adding two 2-bit integers must use that carry. Build the 2-bit adder shown below and make sure it works by testing all combinations of input (again, write the truth table first, so you'll know what to expect). Don't worry about the final carry (it just makes the circuit mode complex). Notice how the half adder is included in this circuit.

Hint: annotate this diagram with which pins on which chips are which connections, to make it obvious which wires should go where when you wire it up.



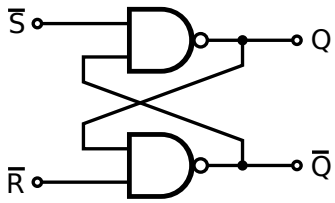
Note: In this diagram, *a* and *b* refer to the two 2-bit numbers being added. Specifically, *a0* is the *ones* bit for *a*, and *a1* is the *twos* bit for *a*. So a=2 (a=10 in binary) would have a1=1, a0=0 . In this diagram, they are grouped by bit to make the diagram simpler. However, when laying out your breadboard, you should group the input jumper wires for each 2-bit number together, with the least-significant bit (the *ones* bit) on the right. Do the same with the output LEDs. That is:

Inputs: a1 a0 b1 b0 Outputs: (a+b)1 (a+b)0

Extra (Optional) Exercise: 2-bit Adder with Carry

Add the carry logic to the previous exercise to create a circuit that adds two 2-bit numbers together to make a 3-bit answer (2 bits plus carry). This can get a bit finicky, but it does demonstrate how hard it is to do electronics on a breadboard. This is an *optional* extra – if it gets too finicky then stop.

Extra (Optional) Exercise: S-R latch using NAND gates



On your breadboard, implement an S-R latch using NAND gates. Note that the inputs are inverting (active-low) due to the use of NAND rather than NOR (which we don't have in the kits). The "Set" operation (triggered by connecting $\overline{S}$ to ground/negative) sets output Q to true, and "Reset" (grounding $\overline{R}$) sets Q to false. $\overline{Q}$ is just a negated copy of Q.

Summary

We can build circuits to do binary addition, and chain them together for arbitrarily large binary integers. You already know how to use 2's complement for negative numbers so it can also do subtraction!